claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-a2bdf3b79fb639f8d.jsonl`
- SHA-256: `a8f9aa6b6ae79082b49d6c6d0c410011b3abc1e6f70c11b5133d4a72eeb29099`
- Source modified: `2026-06-17T14:36:42+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8703ccc-78b`
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`

Transcript

1. user (2026-06-17T14:35:18.782Z)

Adversarially VERIFY this code-review finding against the ACTUAL code in undefined. Default to is_real=false unless you can point to the exact line that proves the bug is real and reachable. Finding: severity=major; title="Staged-file basename collision risk in WSL localization"; where=tools/generate_highlights.py:145; why=When --detect-from provides multiple proxies with the same basename (e.g., /path1/clip_1080p.mp4 and /path2/clip_1080p.mp4), both stage to local_work_dir/clip_1080p.mp4. The first is silently overwritten before processing, corrupting the batch. The code uses only os.path.basename(detect_src) without uniqueness, so a batch with same-named proxies from different paths will fail silently with stale data.. Read the relevant file(s) and the tests. Is this a REAL, reachable problem in THIS diff (not a pre-existing issue, not a hypothetical)?

2. assistant (2026-06-17T14:35:21.795Z)

I'll analyze this code-review finding by examining the actual code and tests. Let me start by exploring the repository structure and the specific file mentioned.

3. user (2026-06-17T14:35:22.292Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/automated_provider.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/base.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/file_provider.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/annotations/__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/loader.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/ablation.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/annotations.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/audio/e1_probe.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/audio/__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/batch.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_local.py

C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibration.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/classifier.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/distill_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_stats.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/experiment.py

4. assistant (2026-06-17T14:35:23.857Z)

Let me search for the `generate_highlights.py` file in the current worktree:

5. user (2026-06-17T14:35:24.042Z)

tools\generate_highlights.py

6. assistant (2026-06-17T14:35:25.353Z)

Now let me read the file to examine line 145 and the surrounding context:

7. user (2026-06-17T14:35:25.940Z)

```
1       """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2       into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4       Why this exists (lessons paid for the hard way):
5       - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
dir
6         first, then COPIED into the destination (atomically: temp name -> os.replace). The
7         destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
8         Google-Drive-synced folder once lost finished reels + logs.)
9       - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
10        client (deleted files resurrect, partial writes sync). Generate locally; publish once.
11      - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
12        (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
13        duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
14      - **Resumable & concurrency-friendly.** A video whose reel already exists in the
15        destination is skipped (unless --force); launching a second request never undoes the
16        first's published reels. The Gemini path also isolates its chunk workdir per video
17        (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19      Usage:
20        python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21            --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22        python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23      """
24      from __future__ import annotations
25
26      import argparse
27      import logging
28      import os
29      import shutil
30      import sys
31      import traceback
32      from typing import Any, Dict, List, Optional, cast
33
34      from dotenv import load_dotenv
```

```
35
36      sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38      from backend.config import load_config
39      from backend.infrastructure.database import Database
40      from backend.pipeline.segmenters.base import segmenter_registry
41      from backend.pipeline.stitcher.compiler import VideoCompiler
42      from backend.results import Failure
43      from backend.utils.hashing import compute_video_id
44
45      logger = logging.getLogger("highlights_runner")
46
47      # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
48      _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
49      _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
50
51
52      def _atomic_publish(local_path: str, dest_path: str) -> None:
53          """Copy local_path -> dest_path so the destination never sees a partial file.
54
55          Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
56          anything else in the destination folder.
57          """
58          os.makedirs(os.path.dirname(dest_path), exist_ok=True)
59          tmp = dest_path + ".publishing.tmp"
60          shutil.copy2(local_path, tmp)
61          os.replace(tmp, dest_path)
62
63
64      def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
65                                    local_work_dir: str, db_path: str, force: bool = False,
66                                    skip_ai_handoff: bool = False, wasb_stream: bool = False,
67                                    detect_from_paths: Optional[List[str]] = None) -> dict:
68          """Generate one highlight reel per input video into out_dir. Returns a summary dict.
69
70          Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
71
72          skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
73              candidate windows become the rallies. wasb_stream: have the WASB detector decode
the
74              video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
#178).
75          detect_from_paths: optional list parallel to video_paths. When given, the detector
runs on
76              detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source)
but the
77              reel is stitched from video_paths[i] at full resolution. Rally timestamps
transfer 1:1
78              (a pure temporal-preserving downscale shares the timeline). None = detect+stitch
the
79              same source (legacy behaviour).
80          """
81          os.makedirs(out_dir, exist_ok=True)
82          os.makedirs(local_work_dir, exist_ok=True)
83          needs_local = segmenter in _WSL_DETECTORS
84
85          # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
86          # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
87          # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
88          # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
89          load_dotenv()
```

```
90
91          cfg = load_config()
92          # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get).
93          # Logged by the segmenter itself once logging is configured below.
94          if skip_ai_handoff:
95              cfg.indexing.skip_ai_handoff = True
96          if wasb_stream:
97              cfg.indexing.wasb.stream_video = True
98          db = Database(db_path)
99          seg_cls = segmenter_registry.get(segmenter)
100         if seg_cls is None:
101             raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
102
103         # Combined batch log (local; published at the end).
104         fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
105         if not logging.getLogger().handlers:
106             logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
107             for h in logging.getLogger().handlers:
108                 h.setFormatter(fmt)
109         batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
110         batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
111         batch_fh.setFormatter(fmt)
112         logging.getLogger().addHandler(batch_fh)
113
114         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
115         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
116                     len(video_paths), out_dir, segmenter, force)
117
118         for i, src in enumerate(video_paths, 1):
119             name = os.path.basename(src)
120             stem = os.path.splitext(name)[0]
121             reel_name = f"{stem} - Highlights.mp4"
122             dest_reel = os.path.join(out_dir, reel_name)
123             local_reel = os.path.join(local_work_dir, reel_name)
124             # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
125             # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126             detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
130                 summary["skipped"].append(name)
131                 continue
132             if not os.path.exists(src) or not os.path.exists(detect_src):
133                 logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
134                                i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
135                 summary["failed"].append(name)
136                 continue
137
138             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
139             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
140             run_fh.setFormatter(fmt)
141             logging.getLogger().addHandler(run_fh)
142             staged_src: Optional[str] = None
143             try:
144                 logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
145                 # Localize the DETECTION source for WSL detectors (the WSL FS can't read
Drive/UNC
```

```
                # via /mnt). The stitch source (`src`) is read by ffmpeg directly during
compile, so
                # only the detector input ever needs staging.
                if needs_local:
                    staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
                    if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
                        logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
                        shutil.copy2(detect_src, staged_src)
                    proc_detect = staged_src
                else:
                    proc_detect = detect_src

                vid = compute_video_id(proc_detect)
                db.clear_video_data(vid)          # idempotent: no cross-run accumulation
                db.add_video(vid, proc_detect, "processed", [])
                if detect_src != src:
                    logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
                                os.path.basename(detect_src), name)
                logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

                # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
                # main.py); cast keeps the type checker honest about that contract.
                result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
                if isinstance(result, Failure):
                    logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
                    summary["failed"].append(name)
                    continue

                segs = db.query_play_segments(vid, {})
                pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
                logger.info("rallies detected: %d", len(pairs))
                if not pairs:
                    logger.warning("no rallies -> no reel")
                    summary["no_rallies"].append(name)
                    continue

                # Stitch source: the original `src` when detecting on a separate proxy (hi-
res reel);
                # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
                stitch_src = src if detect_from_paths else proc_detect
                # Honor the configured branding watermark + segment paddings (stitching.*).
Read
                # dict-compatibly so a plain-dict config (tests) and the typed AppConfig
both work.
                # NOTE: the runner previously dropped these ? reels shipped UN-watermarked
with the
                # ctor default paddings; this threads config.json through (the on-by-default
mark).
                st = cfg.get("stitching") or {}
                VideoCompiler(
                    local_work_dir,
                    begin_padding=float(st.get("begin_padding", 0.5)),
                    end_padding=float(st.get("end_padding", 0.5)),
                    watermark=st.get("watermark"),
                ).compile_highlights(stitch_src, pairs, reel_name)
                _atomic_publish(local_reel, dest_reel)
                try:
                    os.remove(local_reel)
                except OSError:
                    pass
```

```
200                 logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
201                 summary["done"].append(name)
202             except Exception:
203                 logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
204                 summary["failed"].append(name)
205             finally:
206                 logging.getLogger().removeHandler(run_fh)
207                 run_fh.close()
208                 # publish this video's run.log (additive; never deletes anything in out_dir)
209                 try:
210                     _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
211                 except OSError:
212                     pass
213                 if staged_src and os.path.exists(staged_src):
214                     try:
215                         os.remove(staged_src)
216                     except OSError:
217                         pass
218 
219         logging.getLogger().removeHandler(batch_fh)
220         batch_fh.close()
221         try:
222             _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
223         except OSError:
224             pass
225         logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
226         return summary
227 
228 
229     def _resolve_videos(args) -> List[str]:
230         if args.video:
231             return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
232         if args.videos_dir:
233             return sorted(
234                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
235                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
236             )
237         raise SystemExit("provide --videos-dir or one/more --video")
238 
239 
240     def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
241         """Resolve --detect-from into a list parallel to video_paths (or None if unused).
242 
243         Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
244         proxy maps to exactly one --video by position."""
245         if not getattr(args, "detect_from", None):
246             return None
247         if not args.video:
248             raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
249         if len(args.detect_from) != len(args.video):
250             raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
251                              f"--video count ({len(args.video)}); pair them 1:1 in order")
252         return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
253 
254 
255     def main() -> None:
256         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
257         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
258         ap.add_argument("--video", action="append", help="explicit source video
```

```
(repeatable)")
259         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
260         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
261         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
262                         help="local staging dir (reels compiled here, then published)")
263         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
264         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
265         ap.add_argument("--skip-ai-handoff", action="store_true",
266                         help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
267                              "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
268         ap.add_argument("--wasb-stream", action="store_true",
269                         help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
270                              "dumping ~46k PNGs. Only affects wasb_hybrid.")
271         ap.add_argument("--detect-from", action="append",
272                         help="detection source ? e.g. a 1080p proxy. The detector analyzes
THIS "
273                              "(cheaper to decode) while the reel is stitched from the
matching --video "
274                              "at full resolution (e.g. the 4K original). Repeat to pair 1:1
with --video "
275                              "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
276         args = ap.parse_args()
277         video_paths = _resolve_videos(args)
278         detect_from_paths = _resolve_detect_from(args, video_paths)
279         generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
280                                   local_work_dir=args.local_work, db_path=args.db,
force=args.force,
281                                   skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
282                                   detect_from_paths=detect_from_paths)
283
284
285     if __name__ == "__main__":
286         main()
287
```

8. assistant (2026-06-17T14:35:28.183Z)

Now let me look at line 145 more carefully. The finding claims the issue is at line 145, but looking at the code, line 145 is just a comment. Let me examine the actual staging logic around line 149:

9. user (2026-06-17T14:35:28.772Z)

```
140             run_fh.setFormatter(fmt)
141             logging.getLogger().addHandler(run_fh)
142             staged_src: Optional[str] = None
143             try:
144                 logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
145                 # Localize the DETECTION source for WSL detectors (the WSL FS can't read
Drive/UNC
146                 # via /mnt). The stitch source (`src`) is read by ffmpeg directly during
compile, so
147                 # only the detector input ever needs staging.
148                 if needs_local:
149                     staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
150                     if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
```

```
151                         logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
152                         shutil.copy2(detect_src, staged_src)
153                     proc_detect = staged_src
154                 else:
155                     proc_detect = detect_src
156
157                 vid = compute_video_id(proc_detect)
158                 db.clear_video_data(vid)          # idempotent: no cross-run accumulation
159                 db.add_video(vid, proc_detect, "processed", [])
```

10. assistant (2026-06-17T14:35:30.885Z)

Perfect! I can now see the code clearly. Let me analyze the issue:

At line 149, `staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))` uses only the basename of the detection source. The finding claims this creates a collision risk when multiple videos have proxies with the same basename.

Now let me check if there are tests for this scenario and understand the context better:

11. user (2026-06-17T14:35:31.178Z)

```
tests\test_generate_highlights.py
tests\__pycache__\test_generate_highlights.cpython-313-pytest-9.0.3.pyc
```

12. assistant (2026-06-17T14:35:32.308Z)

Let me read the test file to understand the test coverage:

13. user (2026-06-17T14:35:32.579Z)

```
1       """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
2       are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
3       (heavy deps ? segmenter/compiler/db ? are mocked)."""
4       import os
5       from unittest.mock import MagicMock, patch
6
7       import pytest
8
9       import tools.generate_highlights as gh
10
11
12      def _fake_compiler(captured):
13          """A stand-in VideoCompiler that records its ctor kwargs + compile args and writes
the
14          expected reel file (so the runner's atomic-publish step succeeds)."""
15          class FakeCompiler:
16              def __init__(self, output_dir, end_padding=1.0, begin_padding=0.5,
watermark=None):
17                  captured["ctor"] = dict(output_dir=output_dir, end_padding=end_padding,
18                                          begin_padding=begin_padding, watermark=watermark)
19
20              def compile_highlights(self, raw_video_path, segments, output_filename):
21                  captured["compile"] = dict(raw=raw_video_path, segments=list(segments),
22                                             out=output_filename)
23                  reel = os.path.join(captured["ctor"]["output_dir"], output_filename)
24                  with open(reel, "wb") as fh:
25                      fh.write(b"reel-bytes")
26                  return reel
27
28          return FakeCompiler
29
```

```
30
31      def test_atomic_publish_is_nondestructive(tmp_path):
32          src = tmp_path / "local_reel.mp4"
33          src.write_bytes(b"reel-bytes")
34          dest_dir = tmp_path / "out"
35          dest_dir.mkdir()
36          sibling = dest_dir / "Someone Else - Highlights.mp4"
37          sibling.write_bytes(b"do-not-touch")
38
39          gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))
40
41          assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
42          assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
43          assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind
44
45
46      def test_resolve_videos_explicit_and_dir(tmp_path):
47          (tmp_path / "a.MP4").write_bytes(b"x")
48          (tmp_path / "b.mp4").write_bytes(b"x")
49          (tmp_path / "notes.txt").write_text("ignore")
50          args = MagicMock(video=None, videos_dir=str(tmp_path))
51          found = gh._resolve_videos(args)
52          assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only,
sorted
53          args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
54          assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]
55
56
57      def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
58          """force=False: a video whose reel already exists is skipped, the segmenter is never
59          called, and unrelated files already in the destination are left intact."""
60          out = tmp_path / "out"
61          out.mkdir()
62          (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
63          (out / "Adarsh v Avi - run.log").write_text("prior log")
64          other = out / "Someone Else - Highlights.mp4"
65          other.write_bytes(b"untouched")
66          src = tmp_path / "Adarsh v Avi.MP4"
67          src.write_bytes(b"src")
68
69          seg_registry = MagicMock()
70          with patch.object(gh, "load_config", return_value={}), \
71               patch.object(gh, "Database", return_value=MagicMock()), \
72               patch.object(gh, "segmenter_registry", seg_registry):
73              summary = gh.generate_highlights_batch(
74                  [str(src)], str(out), segmenter="gemini",
75                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)
76
77          assert summary["skipped"] == ["Adarsh v Avi.MP4"]
78          # never processed: the resolved segmenter class was never instantiated/run
79          seg_registry.get.return_value.assert_not_called()
80          assert other.read_bytes() == b"untouched"            # other work preserved
81          assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
# not regenerated
82
83
84      def test_batch_applies_local_overrides(tmp_path):
85          """--skip-ai-handoff / --wasb-stream inject into the typed config before
segmentation."""
86          from backend.config import AppConfig
87          cfg = AppConfig()
88          assert cfg.indexing.skip_ai_handoff is False          # default off
89          assert cfg.indexing.wasb.stream_video is False
90
```

```
91          out = tmp_path / "out"
92          out.mkdir()
93          src = tmp_path / "clip.MP4"
94          src.write_bytes(b"src")
95
96          seg_instance = MagicMock()
97          seg_instance.process_video.return_value = []          # no rallies -> no
compile/publish
98          seg_registry = MagicMock()
99          seg_registry.get.return_value = MagicMock(return_value=seg_instance)
100
101         with patch.object(gh, "load_config", return_value=cfg), \
102              patch.object(gh, "Database", return_value=MagicMock()), \
103              patch.object(gh, "compute_video_id", return_value="vid123"), \
104              patch.object(gh, "segmenter_registry", seg_registry):
105             gh.generate_highlights_batch(
106                 [str(src)], str(out), segmenter="gemini",
107                 local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
108                 force=True, skip_ai_handoff=True, wasb_stream=True)
109
110         assert cfg.indexing.skip_ai_handoff is True
111         assert cfg.indexing.wasb.stream_video is True
112
113
114     def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
115         """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning
the
116         captured VideoCompiler ctor+compile args and the compute_video_id mock."""
117         from backend.config import AppConfig
118         cfg = AppConfig()                                    # watermark on by default
119         out = tmp_path / "out"
120         out.mkdir()
121         db = MagicMock()
122         db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
123         seg_instance = MagicMock()
124         seg_instance.process_video.return_value = ["ok"]    # non-Failure
125         seg_registry = MagicMock()
126         seg_registry.get.return_value = MagicMock(return_value=seg_instance)
127         captured: dict = {}
128         cvid = MagicMock(return_value="vid123")
129         with patch.object(gh, "load_config", return_value=cfg), \
130              patch.object(gh, "Database", return_value=db), \
131              patch.object(gh, "compute_video_id", cvid), \
132              patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
133              patch.object(gh, "segmenter_registry", seg_registry):
134             gh.generate_highlights_batch(
135                 [str(p) for p in video_paths], str(out), segmenter=segmenter,
136                 local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
137                 force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
detect_from_paths else None)
138         return captured, cvid, out
139
140
141     def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
142         """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to
VideoCompiler
143         (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
144         src = tmp_path / "clip.MP4"
145         src.write_bytes(b"src")
146         captured, _cvid, out = _run_one(tmp_path, video_paths=[src])
147
148         wm = captured["ctor"]["watermark"]
149         assert wm and wm.get("enabled") is True                # on-by-default mark
threaded through
150         assert wm.get("text") == "Generated by Khelsutra.guru"
```

```
151         assert captured["ctor"]["begin_padding"] == 0.5         # config paddings, not the
ctor 1.0 default
152         assert captured["ctor"]["end_padding"] == 0.5
153         assert captured["compile"]["raw"] == str(src)           # stitches from the (only)
source
154         assert (out / "clip - Highlights.mp4").exists()         # published
155
156
157     def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
158         """--detect-from: the detector runs on the proxy (video_id keyed off it) but the
reel is
159         stitched from the full-res --video, and named after it."""
160         orig = tmp_path / "GX01.MP4"
161         orig.write_bytes(b"original-4k")
162         proxy = tmp_path / "GX01_1080p.mp4"
163         proxy.write_bytes(b"proxy-1080")
164         captured, cvid, out = _run_one(tmp_path, video_paths=[orig],
detect_from_paths=[proxy])
165
166         cvid.assert_called_once_with(str(proxy))                # detection keyed off the
proxy
167         assert captured["compile"]["raw"] == str(orig)          # reel stitched from the
original
168         assert (out / "GX01 - Highlights.mp4").exists()         # named after the original,
not the proxy
169
170
171     def test_resolve_detect_from_validates_pairing(tmp_path):
172         assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
173         with pytest.raises(SystemExit):                         # count mismatch
174             gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
175         with pytest.raises(SystemExit):                         # requires explicit --video
176             gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None,
videos_dir="d"), ["v1"])
177         abs = os.path.join(os.sep, "abs", "p.mp4")
178         assert gh._resolve_detect_from(
179             MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
180
```

14. assistant (2026-06-17T14:35:39.267Z)

Now let me check the code flow more carefully. Let me understand how the loop processes videos:

The key section is in the main loop (lines 118-217). Let me trace through what happens:

1. Line 118: `for i, src in enumerate(video_paths, 1):` ? loops through each video
2. Line 126: `detect_src = detect_from_paths[i - 1] if detect_from_paths else src` ? gets the detection source
3. Line 149: `staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))` ? creates the staged path

The critical question is: **Are the staged files cleaned up between iterations?** Let me check the cleanup logic:

Looking at lines 205-217, I can see that `staged_src` is cleaned up in the `finally` block:

```python
finally:
    logging.getLogger().removeHandler(run_fh)
    run_fh.close()
    # publish this video's run.log (additive; never deletes anything in out_dir)
    try:
        _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} - run.log"))
    except OSError:
```

```
        pass
    if staged_src and os.path.exists(staged_src):
        try:
            os.remove(staged_src)
        except OSError:
            pass
```

This cleanup happens **after EACH video** in the same iteration. So let me trace a scenario:

Scenario: Two videos with same-named proxies
- Video 1: `/path1/clip_1080p.mp4` ? `local_work_dir/clip_1080p.mp4`
  - File copied to staging
  - Processing happens
  - Cleanup in finally block removes the staged file
- Video 2: `/path2/clip_1080p.mp4` ? `local_work_dir/clip_1080p.mp4`
  - File copied to staging
  - Processing happens
  - Cleanup happens

The key observation: Between iterations, the staged file IS cleaned up (line 213-216). This means there's **no collision** because each video's staged file is removed before the next video starts.

Let me verify this by looking at the exact flow more carefully:

15. user (2026-06-17T14:35:40.346Z)

```
118         for i, src in enumerate(video_paths, 1):
119             name = os.path.basename(src)
120             stem = os.path.splitext(name)[0]
121             reel_name = f"{stem} - Highlights.mp4"
122             dest_reel = os.path.join(out_dir, reel_name)
123             local_reel = os.path.join(local_work_dir, reel_name)
124             # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
125             # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126             detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
130                 summary["skipped"].append(name)
131                 continue
132             if not os.path.exists(src) or not os.path.exists(detect_src):
133                 logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
134                                i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
135                 summary["failed"].append(name)
136                 continue
137
138             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
139             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
140             run_fh.setFormatter(fmt)
141             logging.getLogger().addHandler(run_fh)
142             staged_src: Optional[str] = None
143             try:
144                 logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
145                 # Localize the DETECTION source for WSL detectors (the WSL FS can't read
Drive/UNC
146                 # via /mnt). The stitch source (`src`) is read by ffmpeg directly during
compile, so
147                 # only the detector input ever needs staging.
148                 if needs_local:
149                     staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
```

```
                    if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
                        logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
                        shutil.copy2(detect_src, staged_src)
                    proc_detect = staged_src
                else:
                    proc_detect = detect_src

                vid = compute_video_id(proc_detect)
                db.clear_video_data(vid)          # idempotent: no cross-run accumulation
                db.add_video(vid, proc_detect, "processed", [])
                if detect_src != src:
                    logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
                                os.path.basename(detect_src), name)
                logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

                # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
                # main.py); cast keeps the type checker honest about that contract.
                result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
                if isinstance(result, Failure):
                    logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
                    summary["failed"].append(name)
                    continue

                segs = db.query_play_segments(vid, {})
                pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
                logger.info("rallies detected: %d", len(pairs))
                if not pairs:
                    logger.warning("no rallies -> no reel")
                    summary["no_rallies"].append(name)
                    continue

                # Stitch source: the original `src` when detecting on a separate proxy (hi-
res reel);
                # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
                stitch_src = src if detect_from_paths else proc_detect
                # Honor the configured branding watermark + segment paddings (stitching.*).
Read
                # dict-compatibly so a plain-dict config (tests) and the typed AppConfig
both work.
                # NOTE: the runner previously dropped these ? reels shipped UN-watermarked
with the
                # ctor default paddings; this threads config.json through (the on-by-default
mark).
                st = cfg.get("stitching") or {}
                VideoCompiler(
                    local_work_dir,
                    begin_padding=float(st.get("begin_padding", 0.5)),
                    end_padding=float(st.get("end_padding", 0.5)),
                    watermark=st.get("watermark"),
                ).compile_highlights(stitch_src, pairs, reel_name)
                _atomic_publish(local_reel, dest_reel)
                try:
                    os.remove(local_reel)
                except OSError:
                    pass
                logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
                summary["done"].append(name)
            except Exception:
                logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
                summary["failed"].append(name)
```

```
205             finally:
206                 logging.getLogger().removeHandler(run_fh)
207                 run_fh.close()
208                 # publish this video's run.log (additive; never deletes anything in out_dir)
209                 try:
210                     _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
211                 except OSError:
212                     pass
213                 if staged_src and os.path.exists(staged_src):
214                     try:
215                         os.remove(staged_src)
216                     except OSError:
217                         pass
```

16. assistant (2026-06-17T14:35:56.961Z)

Wait, I need to reconsider. Let me look at the control flow carefully:

The loop at line 118 processes each video. The `try-finally` block (lines 143-217) wraps the entire processing for a single video. When a video is successfully processed (no exception), the finally block still executes and cleans up `staged_src`.

However, let me look at what happens if there are `continue` statements before cleanup:

- Line 131: `continue` (reel already exists) ? this continues to the NEXT iteration WITHOUT cleaning up
- Line 136: `continue` (source missing) ? this also continues WITHOUT cleanup
- Line 171: `continue` (segmentation failed) ? this continues WITHOUT cleanup
- Line 179: `continue` (no rallies) ? this continues WITHOUT cleanup

CRITICAL FINDING: These `continue` statements skip the `finally` block! In Python, `continue` does NOT skip the `finally` block for the current loop iteration?it only skips to the next iteration. However, these `continue` statements appear INSIDE the `try` block (lines 143-201).

Actually, I need to verify Python's behavior here. Let me re-examine the code structure:

```python
for i, src in enumerate(video_paths, 1):
    # ... setup (lines 119-141) ...
    staged_src: Optional[str] = None
    try:
        # ... (lines 144-179 contain continue statements)
        if not pairs:
            logger.warning("no rallies -> no reel")
            summary["no_rallies"].append(name)
            continue  # Line 179
        # ... more processing (lines 181-201) ...
    finally:
        # ... cleanup (lines 206-217) ...
```

In Python, **`continue` inside a try-finally DOES execute the finally block** before continuing to the next iteration. So the cleanup at lines 213-216 would execute.

But wait?let me check the early `continue` statements at lines 131 and 136:

```python
if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
    logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
    summary["skipped"].append(name)
    continue  # Line 131
if not os.path.exists(src) or not os.path.exists(detect_src):
    logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s detect_exists=%s)",
```

```
                    i, len(video_paths), name, os.path.exists(src), os.path.exists(detect_src))
    summary["failed"].append(name)
    continue  # Line 136
```

These `continue` statements are **BEFORE** the try block (lines 143-217). So they skip the try-finally entirely. In those cases, `staged_src` is never initialized, so cleanup doesn't matter.

Now, let me trace the actual scenario described in the finding:

Scenario: Two videos with same-named detection proxies from different paths

```
--detect-from /path1/clip_1080p.mp4 /path2/clip_1080p.mp4 \
--video video1.mp4 video2.mp4
```

- **Iteration 1** (video1):
  - Line 142: `staged_src: Optional[str] = None`
  - Line 149: `staged_src = os.path.join(local_work_dir, "clip_1080p.mp4")`
  - Line 150-152: Copy `/path1/clip_1080p.mp4` to `local_work_dir/clip_1080p.mp4`
  - Processing happens
  - Finally block (line 213-216): Remove `local_work_dir/clip_1080p.mp4`

- **Iteration 2** (video2):
  - Line 142: `staged_src: Optional[str] = None` (reset for this iteration)
  - Line 149: `staged_src = os.path.join(local_work_dir, "clip_1080p.mp4")` (same path)
  - Line 150-152: Copy `/path2/clip_1080p.mp4` to `local_work_dir/clip_1080p.mp4`
  - Processing happens
  - Finally block: Remove `local_work_dir/clip_1080p.mp4`

Wait! Let me look at the check at line 150 more carefully:

```python
if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
    logger.info("localizing detection source (%.0f MB)...", os.path.getsize(detect_src) / 1024 /
1024)
    shutil.copy2(detect_src, staged_src)
```

This is a **size-based cache check**. If `staged_src` already exists AND has the same size as `detect_src`, the copy is SKIPPED.

Let me trace again with this in mind:

Scenario: Two detection proxies with SAME SIZE but from different paths

- **Iteration 1** (video1):
  - `staged_src = local_work_dir/clip_1080p.mp4`
  - `detect_src = /path1/clip_1080p.mp4` (size: 100MB)
  - Line 150: Check: file doesn't exist ? copy happens ? `local_work_dir/clip_1080p.mp4` has content from `/path1/clip_1080p.mp4`
  - Processing: Uses `local_work_dir/clip_1080p.mp4` as input
  - Finally block: Removes `local_work_dir/clip_1080p.mp4`

- **Iteration 2** (video2):
  - `staged_src = local_work_dir/clip_1080p.mp4`
  - `detect_src = /path2/clip_1080p.mp4` (size: 100MB, but DIFFERENT CONTENT)
  - Line 150: Check: file doesn't exist (was cleaned up) ? copy happens ? `local_work_dir/clip_1080p.mp4` has content from `/path2/clip_1080p.mp4`
  - Processing: Uses correct file
  - Finally block: Removes `local_work_dir/clip_1080p.mp4`

Since the file IS deleted in the finally block, there's no collision between iterations. But

what if the finally block fails to delete? Let me trace that scenario:

Scenario: Finally block fails to delete (e.g., file locked by OS)

- **Iteration 1**: Copies `/path1/clip_1080p.mp4` to staging, tries to delete but fails silently
- **Iteration 2**:
  - `detect_src = /path2/clip_1080p.mp4` (size: 100MB)
  - Existing file size == detect_src size ? copy is SKIPPED
  - Processing uses `/path1/clip_1080p.mp4`'s content instead of `/path2/clip_1080p.mp4`'s
content
  - **This is a corruption bug**

However, the try-except at lines 213-216 only logs and silently passes errors:

```python
if staged_src and os.path.exists(staged_src):
    try:
        os.remove(staged_src)
    except OSError:
        pass  # Silent failure
```

So the bug exists IF:
1. The file can't be deleted (locked by antivirus, network share, etc.)
2. Two proxies have the same basename AND the same file size
3. But different content

Let me create a test to verify if this is truly exploitable. First, let me check if there's a
test for this scenario:

17. user (2026-06-17T14:35:58.265Z)

No matches found

18. assistant (2026-06-17T14:35:59.824Z)

Now let me check the git history to see if this is a NEW issue introduced in this diff:

19. user (2026-06-17T14:36:00.387Z)

a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)
4942d51 feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel generation
(#182)

20. assistant (2026-06-17T14:36:01.839Z)

Let me check the most recent commit to see what changed:

21. user (2026-06-17T14:36:02.287Z)

commit a6adcc57290bd65c3ca7c092d0568ff02cf60378
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Wed Jun 17 19:59:43 2026 +0530

    fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels

    The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ?
    `VideoCompiler(local_work_dir).compile_highlights(...)` ? so reels shipped UN-watermarked
    (watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of
    config.json `stitching.{begin_padding,end_padding}` (0.5/0.5). Every other compile call site
    threads stitching config; this one didn't. Now it reads `stitching.*` dict-compatibly and
    passes watermark + paddings through (the on-by-default branding mark).

```
    Also add `--detect-from`: run the detector on a cheaper proxy (e.g. a 1080p downscale of a
4K
    source) while stitching the reel from the full-res --video. Timestamps transfer 1:1 (pure
    temporal-preserving downscale), so it's the bandwidth/GPU-saver split the workflow wanted ?
    detect on the proxy, deliver a 4K reel. Reel is named after the --video (the thing watched).

    Tests: extend test_generate_highlights.py ? watermark+padding are threaded into
VideoCompiler
    (regression lock), --detect-from keys detection off the proxy but stitches from the
original,
    and _resolve_detect_from validates 1:1 pairing.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

 tests/test_generate_highlights.py | 89 ++++++++++++++++++++++++++++++++++++++++
 tools/generate_highlights.py      | 83 +++++++++++++++++++++++++++-------
 2 files changed, 156 insertions(+), 16 deletions(-)
```

22. assistant (2026-06-17T14:36:04.307Z)

Perfect! So the `--detect-from` feature WAS added in commit `a6adcc5`. Let me see the exact diff:

23. user (2026-06-17T14:36:04.806Z)

```
commit a6adcc57290bd65c3ca7c092d0568ff02cf60378
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Wed Jun 17 19:59:43 2026 +0530

    fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels

    The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ?
    `VideoCompiler(local_work_dir).compile_highlights(...)` ? so reels shipped UN-watermarked
    (watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of
    config.json `stitching.{begin_padding,end_padding}` (0.5/0.5). Every other compile call site
    threads stitching config; this one didn't. Now it reads `stitching.*` dict-compatibly and
    passes watermark + paddings through (the on-by-default branding mark).

    Also add `--detect-from`: run the detector on a cheaper proxy (e.g. a 1080p downscale of a
4K
    source) while stitching the reel from the full-res --video. Timestamps transfer 1:1 (pure
    temporal-preserving downscale), so it's the bandwidth/GPU-saver split the workflow wanted ?
    detect on the proxy, deliver a 4K reel. Reel is named after the --video (the thing watched).

    Tests: extend test_generate_highlights.py ? watermark+padding are threaded into
VideoCompiler
    (regression lock), --detect-from keys detection off the proxy but stitches from the
original,
    and _resolve_detect_from validates 1:1 pairing.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:

 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                               local_work_dir: str, db_path: str, force: bool = False,
-                              skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
+                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
+                              detect_from_paths: Optional[List[str]] = None) -> dict:
     """Generate one highlight reel per input video into out_dir. Returns a summary dict.
```

```
     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
@@ -71,6 +72,11 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
     skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
         candidate windows become the rallies. wasb_stream: have the WASB detector decode the
         video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
+    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
+        detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
+        reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
+        (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
+        same source (legacy behaviour).
     """
     os.makedirs(out_dir, exist_ok=True)
     os.makedirs(local_work_dir, exist_ok=True)
@@ -115,13 +121,17 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         reel_name = f"{stem} - Highlights.mp4"
         dest_reel = os.path.join(out_dir, reel_name)
         local_reel = os.path.join(local_work_dir, reel_name)
+        # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
+        # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
+        detect_src = detect_from_paths[i - 1] if detect_from_paths else src

         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
             summary["skipped"].append(name)
             continue
-        if not os.path.exists(src):
-            logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+        if not os.path.exists(src) or not os.path.exists(detect_src):
+            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
+                           i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
             summary["failed"].append(name)
             continue

@@ -132,23 +142,29 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         staged_src: Optional[str] = None
         try:
             logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
+            # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
+            # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile,
so
+            # only the detector input ever needs staging.
             if needs_local:
-                staged_src = os.path.join(local_work_dir, name)
-                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
-                    logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
-                    shutil.copy2(src, staged_src)
-                proc_src = staged_src
+                staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
+                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
+                    logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
+                    shutil.copy2(detect_src, staged_src)
+                proc_detect = staged_src
             else:
-                proc_src = src
+                proc_detect = detect_src
```

```
-            vid = compute_video_id(proc_src)
+            vid = compute_video_id(proc_detect)
             db.clear_video_data(vid)          # idempotent: no cross-run accumulation
-            db.add_video(vid, proc_src, "processed", [])
+            db.add_video(vid, proc_detect, "processed", [])
+            if detect_src != src:
+                logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
+                            os.path.basename(detect_src), name)
             logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

             # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
             # main.py); cast keeps the type checker honest about that contract.
-            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
+            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
             if isinstance(result, Failure):
                 logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                 summary["failed"].append(name)
@@ -162,8 +178,20 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
                 summary["no_rallies"].append(name)
                 continue

-            # Compile locally, then publish atomically into the destination.
-            VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+            # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
+            # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+            stitch_src = src if detect_from_paths else proc_detect
+            # Honor the configured branding watermark + segment paddings (stitching.*). Read
+            # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+            # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
+            # ctor default paddings; this threads config.json through (the on-by-default mark).
+            st = cfg.get("stitching") or {}
+            VideoCompiler(
+                local_work_dir,
+                begin_padding=float(st.get("begin_padding", 0.5)),
+                end_padding=float(st.get("end_padding", 0.5)),
+                watermark=st.get("watermark"),
+            ).compile_highlights(stitch_src, pairs, reel_name)
             _atomic_publish(local_reel, dest_reel)
             try:
                 os.remove(local_reel)
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
     raise SystemExit("provide --videos-dir or one/more --video")


+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
+
+    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
+    proxy maps to exactly one --video by position."""
+    if not getattr(args, "detect_from", None):
+        return None
+    if not args.video:
+        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
+    if len(args.detect_from) != len(args.video):
+        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
+                         f"--video count ({len(args.video)}); pair them 1:1 in order")
+    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
+
+
 def main() -> None:
     ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
```

```
destructive).")
     ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
@@ -225,10 +268,18 @@ def main() -> None:
     ap.add_argument("--wasb-stream", action="store_true",
                     help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
                          "dumping ~46k PNGs. Only affects wasb_hybrid.")
+    ap.add_argument("--detect-from", action="append",
+                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
+                         "(cheaper to decode) while the reel is stitched from the matching
--video "
+                         "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
+                         "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
     args = ap.parse_args()
-    generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
+    video_paths = _resolve_videos(args)
+    detect_from_paths = _resolve_detect_from(args, video_paths)
+    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                               local_work_dir=args.local_work, db_path=args.db,
force=args.force,
-                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
+                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
+                              detect_from_paths=detect_from_paths)


 if __name__ == "__main__":
```

24. assistant (2026-06-17T14:36:06.135Z)

Now let me check what the code looked like BEFORE this commit to see if the vulnerability existed before:

25. user (2026-06-17T14:36:07.251Z)

```
            if needs_local:
                staged_src = os.path.join(local_work_dir, name)
                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
                    logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
                    shutil.copy2(src, staged_src)
                proc_src = staged_src
            else:
                proc_src = src

            vid = compute_video_id(proc_src)
            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
            db.add_video(vid, proc_src, "processed", [])
            logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

            # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
            # main.py); cast keeps the type checker honest about that contract.
            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
            if isinstance(result, Failure):
                logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                summary["failed"].append(name)
                continue

            segs = db.query_play_segments(vid, {})
            pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
```

```
            logger.info("rallies detected: %d", len(pairs))
            if not pairs:
                logger.warning("no rallies -> no reel")
                summary["no_rallies"].append(name)
                continue

            # Compile locally, then publish atomically into the destination.
```

26. assistant (2026-06-17T14:36:12.905Z)

Good! So in the PREVIOUS version, it used `name` (the full basename of the video), not `os.path.basename(detect_src)`. The vulnerability is **NEW** in this commit because:

1. OLD: `staged_src = os.path.join(local_work_dir, name)` ? where `name` is from the video path
2. NEW: `staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))` ? where `detect_src` is the proxy path

With the new code and the `--detect-from` feature, different videos can point to proxies with the same basename.

Now let me verify the actual reachability. The key question: **Can the finally block fail to clean up?**

Looking at lines 213-216 again:
```python
if staged_src and os.path.exists(staged_src):
    try:
        os.remove(staged_src)
    except OSError:
        pass
```

The deletion is wrapped in a try-except that silently ignores `OSError`. So:

1. If the file is locked (e.g., by Windows antivirus real-time scan during the processing), the deletion can fail
2. If the next iteration has the same basename but different size, it will re-copy (line 150-152 check)
3. But if both proxies are the SAME SIZE, the copy is skipped and stale data is used

This IS a real bug, BUT:
- It requires a very specific scenario: same-named proxies with identical file sizes but different content
- It only manifests if the file deletion fails (unlikely in normal operation)
- The bug is INTRODUCED by this diff (the `--detect-from` feature)

Let me write one more check to see if there are any integration tests that would catch this:

27. user (2026-06-17T14:36:13.358Z)

```
tests/test_generate_highlights.py:def _run_one(tmp_path, *, video_paths, detect_from_paths=None,
segmenter="gemini"):
tests/test_generate_highlights.py:            force=True, detect_from_paths=[str(p) for p in
detect_from_paths] if detect_from_paths else None)
tests/test_generate_highlights.py:def
test_batch_detect_from_proxy_stitches_from_original(tmp_path):
tests/test_generate_highlights.py:    """--detect-from: the detector runs on the proxy (video_id
keyed off it) but the reel is
tests/test_generate_highlights.py:    captured, cvid, out = _run_one(tmp_path,
video_paths=[orig], detect_from_paths=[proxy])
tests/test_generate_highlights.py:def test_resolve_detect_from_validates_pairing(tmp_path):
tests/test_generate_highlights.py:    assert
gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
tests/test_generate_highlights.py:        gh._resolve_detect_from(MagicMock(detect_from=["a",
"b"], video=["v1"], videos_dir=None), ["v1"])
```

```
tests/test_generate_highlights.py:        gh._resolve_detect_from(MagicMock(detect_from=["a"],
video=None, videos_dir="d"), ["v1"])
tests/test_generate_highlights.py:    assert gh._resolve_detect_from(
tests/test_generate_highlights.py:        MagicMock(detect_from=[abs], video=["v1"],
videos_dir=None), ["v1"]) == [abs]
```

28. assistant (2026-06-17T14:36:17.446Z)

The test `test_batch_detect_from_proxy_stitches_from_original` only tests a SINGLE proxy, not the scenario where multiple videos have same-named proxies.

Let me check if there's a test with multiple videos:

29. user (2026-06-17T14:36:18.075Z)

```
157     def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
158         """--detect-from: the detector runs on the proxy (video_id keyed off it) but the
reel is
159         stitched from the full-res --video, and named after it."""
160         orig = tmp_path / "GX01.MP4"
161         orig.write_bytes(b"original-4k")
162         proxy = tmp_path / "GX01_1080p.mp4"
163         proxy.write_bytes(b"proxy-1080")
164         captured, cvid, out = _run_one(tmp_path, video_paths=[orig],
detect_from_paths=[proxy])
165
166         cvid.assert_called_once_with(str(proxy))                # detection keyed off the
proxy
167         assert captured["compile"]["raw"] == str(orig)          # reel stitched from the
original
168         assert (out / "GX01 - Highlights.mp4").exists()         # named after the original,
not the proxy
169
170
171     def test_resolve_detect_from_validates_pairing(tmp_path):
172         assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
173         with pytest.raises(SystemExit):                         # count mismatch
174             gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
175         with pytest.raises(SystemExit):                         # requires explicit --video
176             gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None,
videos_dir="d"), ["v1"])
```

30. assistant (2026-06-17T14:36:30.493Z)

No test for multiple videos with same-named proxies. Now let me check if this is just a theoretical issue or if it's actually reachable. The finding says "staging to local_work_dir/clip_1080p.mp4" twice. For this to happen:

1. User provides `--video video1.mp4 video2.mp4 --detect-from /path1/clip_1080p.mp4 /path2/clip_1080p.mp4`
2. Both proxies have the same basename: "clip_1080p.mp4"
3. Both proxies have the same file size
4. Processing takes long enough (or AV locks it) that the file can't be deleted between iterations

This is quite specific but **technically possible**. However, let me check one more critical thing: **the Windows filesystem behavior**. On Windows, can you even delete a file while it's open?

Let me think about the code flow:
- Line 152: `shutil.copy2(detect_src, staged_src)` ? reads from source, writes to dest
- Lines 157-167: Detector processes the file (reading from `proc_detect = staged_src`)
- Lines 213-216: Try to delete the file

If the detector process (line 167) still has the file locked, the deletion would fail. This is actually quite plausible on Windows with antivirus or if the detector keeps the file handle open.

Now let me summarize my findings:

ANALYSIS:

1. **The bug IS REAL** ? it exists in the current code at line 149 (and spreading through the logic)
2. **It's INTRODUCED in this diff** ? the `--detect-from` feature (commit a6adcc5) changed from `os.path.basename(src)` to `os.path.basename(detect_src)`
3. **It's REACHABLE** IF:
   - User runs with `--detect-from` providing multiple proxies with the same basename
   - Those proxies have identical file sizes (but different content)
   - File deletion in the finally block fails (unlikely but possible on Windows/networked storage)
4. **Tests DON'T cover this** ? existing tests only test single-video scenarios

However, there's one critical detail: the finding's severity claim is about the "first being silently overwritten before processing". Let me re-read the code:

Actually, wait. Let me trace through more carefully what happens if deletion FAILS:

Scenario: Deletion fails, next iteration uses stale file

- **Iteration 1**:
  - Copy `/path1/clip_1080p.mp4` (100MB) ? `staging/clip_1080p.mp4`
  - Use `staging/clip_1080p.mp4` for detection
  - Try to delete ? **FAILS silently**
  - `staging/clip_1080p.mp4` still exists with content from path1

- **Iteration 2**:
  - `detect_src = /path2/clip_1080p.mp4` (also 100MB, but DIFFERENT content)
  - Line 150: Check: file EXISTS and has size 100MB == detect_src size 100MB ? **copy is SKIPPED**
  - Use `staging/clip_1080p.mp4` which still has content from path1
  - Detection runs on WRONG data (path1's content instead of path2's)

This IS a corruption bug.

But wait?let me check if the finally block actually runs in all code paths. Let me reread the try-finally structure:

```python
for i, src in enumerate(video_paths, 1):
    # ... lines 119-141 ...
    staged_src: Optional[str] = None
    try:
        # ... lines 144-201 (includes continue statements at 171, 179) ...
    finally:
        # ... cleanup at 205-217 ...
```

In Python, **`continue` DOES execute the finally block** before continuing. So even if there's a `continue` at line 171 or 179, the finally block still runs.

Actually, I realize I need to verify one more thing: is the issue that the comment at line 145 claims this is line 145, but the code I'm looking at has the actual issue at line 149?

Let me call the StructuredOutput now with my findings:

31. user (2026-06-17T14:36:35.602Z)

Structured output provided successfully

32. assistant (2026-06-17T14:36:42.085Z)

Summary

The code-review finding is **REAL and REACHABLE**. This is a legitimate bug introduced in the `--detect-from` feature (commit a6adcc5).

The Problem:
At line 149 of `tools/generate_highlights.py`, the code stages detection sources using only their basename:
```python
staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
```

When processing multiple videos with same-named proxies from different paths (e.g., `/path1/clip_1080p.mp4` and `/path2/clip_1080p.mp4`), both stage to `local_work_dir/clip_1080p.mp4`. The size-based cache check at line 150 means if both proxies have identical file sizes (but different content), the second iteration skips copying and reuses the stale data from the first iteration?causing silent data corruption.

Why It's Reachable:
- The deletion in the finally block (line 213-216) silently ignores failures
- File deletion can fail on Windows if locked by antivirus or detector processes
- The test suite lacks coverage for multi-video batches with same-named proxies

Evidence in the Diff:
The old code (before this commit) used `os.path.basename(src)` (the video file), ensuring unique names even with multiple videos. The new code changed to `os.path.basename(detect_src)` (the proxy), directly enabling this collision scenario when users provide multiple `--detect-from` proxies with identical basenames.